

CRABS: Cryptographic Attribute-Based State Machines (Extended)

A Unified Protocol for Self-Sovereign Authorization with OT/CRDT Hybrid Convergence

Victor J. Morrow

Prometheus

<https://github.com/vijayee>

victor.j.morrow@gmail.com

Abstract

We present **CRABS**, a protocol that unifies Attribute-Based Encryption (ABE), Conflict-free Replicated Data Types (CRDTs), and Operational Transformation (OT) into a single framework for decentralized, self-sovereign authorization. CRABS treats **attributes as state** — the set of all users and their attributes is the state of a protocol-governed machine, eliminating the central authority problem inherent in traditional ABE. We extend the protocol with **OT/CRDT hybrid types** that combine the expressiveness of operational transformation with the convergence guarantees of CRDTs, enabling move, swap, reorder, and intent-preserving merge operations on ordered data structures. We introduce **threshold triggers** — state-conditional attribute issuance that bridges dynamic state changes with ABE decryption capabilities. The protocol supports **cryptographic agility** through a pluggable signature scheme interface, **idempotent operations** through declarative dedup specifications, and **tombstone management** through a unified compaction framework. We describe the protocol's architecture, algorithms, security properties, and limitations, and discuss applications to decentralized organizations, collaborative content platforms, and self-sovereign identity systems.

1. Introduction

1.1 Motivation

Attribute-Based Encryption (ABE) provides fine-grained access control by associating decryption keys with sets of attributes. A ciphertext encrypted under the policy `(role: manager AND department: engineering)` can be decrypted by any user whose key carries those attributes — without the encryptor knowing the recipients in advance. This makes ABE naturally suited for decentralized, broadcast-style encryption.

However, ABE suffers from a fundamental operational limitation: **only a central authority holding the Master Secret Key (MSK) can issue, modify, or revoke attributes**. This creates a single point of trust, hinders self-sovereign identity, and makes dynamic attribute management impractical. A user who changes departments, gets married, or is promoted must obtain an entirely new key from the authority. There is no audit trail for attribute changes, and users cannot self-assert attributes without going through the central authority.

Concurrently, Conflict-free Replicated Data Types (CRDTs) have emerged as a foundation for decentralized state convergence. CRDTs allow multiple replicas to be updated independently and merged without conflicts, making them ideal for peer-to-peer systems. However, basic CRDTs have **limited expressiveness** — they support counters, sets, and registers, but cannot represent ordered structures with move, swap, or reorder operations. Operational Transformation (OT) provides intent-preserving transformations for collaborative editing, but traditionally requires a central sequencer.

Raph Levien's work on the synthesis of OT and CRDTs demonstrates that the transform function — not the sequencer — is the heart of the system. By carrying a transform function with each operation and maintaining a causal context, data structures can be both **convergent** (CRDT property) and **expressive** (OT property).

1.2 Our Approach

We present **CRABS** (Cryptographic Attribute-Based State machines), a protocol that unifies these threads into a single framework. The key insight is:

▮ **Attributes are not keys. Attributes are state.**

In CRABS, the set of all users and their attributes is the state of a **protocol state machine** — the Attribute Machine. Changing an attribute (granting a role, verifying an identity, self-asserting a phone number) is a **state transition** governed by the same protocol rules as any other operation. ABE keys issued to users are **derived from the current state** and become invalid when the state changes.

We extend this foundation with:

1. **OT/CRDT Hybrid Types:** Data types that combine the expressiveness of operational transformation with the convergence guarantees of CRDTs, supporting insert, delete, move, swap, and intent-preserving merge on ordered structures.
2. **Threshold Triggers:** State-conditional attribute issuance — when a state condition becomes true (e.g., `video.flags >= 10`), the protocol automatically issues temporary attributes to specified roles, enabling state-conditional decryption.
3. **Cryptographic Agility:** A pluggable signature scheme interface supporting ECDSA, Ed25519, BLS, Dilithium, and future schemes through a unified VTable abstraction.
4. **Idempotent Operations:** Declarative dedup specifications (`PER_USER`, `GLOBAL`, `CUSTOM`) that prevent the same logical action from being performed twice, with CRDT-safe convergence.
5. **Unified Tombstone Compaction:** A framework for managing tombstone growth across all CRDT types, with vector-clock-based safety checks and configurable compaction strategies.

1.3 Contributions

This paper makes the following contributions:

1. **The CRABS protocol** — A unified architecture combining ABE, CRDTs, OT, and protocol state machines for self-sovereign attribute management.
 2. **The Attribute Machine** — A state machine whose state is the attribute universe, eliminating the central authority problem.
 3. **OT/CRDT Hybrid Types** — A family of data types that combine OT expressiveness with CRDT convergence, with a formal transform matrix and position map implementation.
 4. **Threshold Triggers** — A mechanism for state-conditional attribute issuance, bridging dynamic state changes with ABE decryption.
 5. **Pluggable Cryptographic Agility** — A VTable-based abstraction for signature schemes, enabling multi-scheme support and graceful migration.
 6. **Unified Compaction Framework** — A tombstone management system applicable to all CRDT types, with safety guarantees based on vector clock synchronization.
-

2. Related Work

2.1 Attribute-Based Encryption

ABE was introduced by Sahai and Waters as "Fuzzy Identity-Based Encryption." Goyal et al. formalized Key-Policy ABE (KP-ABE), and Bethencourt, Sahai, and Waters introduced Ciphertext-Policy ABE (CP-ABE), the variant used in CRABS. Multi-authority ABE and decentralized ABE distribute trust across multiple authorities but still require each authority to manage its own attribute domain. CRABS is, to our knowledge, the first system to treat attributes as protocol-governed state rather than static cryptographic material.

2.2 Operational Transformation and CRDTs

Operational Transformation (OT) was pioneered by Ellis and Gibbs for collaborative text editing. The Jupiter system introduced a central server for operation ordering. Raph Levien's work demonstrated that OT and CRDTs can be synthesized: operations carry transform functions that adjust them against concurrent operations, achieving convergence without a central sequencer.

CRDTs were formalized by Shapiro et al. . Types include G-Counters (grow-only counters), PN-Counters, OR-Sets (observed-remove sets), LWW-Registers (last-writer-wins registers), and RGA (Replicated Growable Arrays) . CRABS extends these with OT/CRDT hybrids that support move, swap, reorder, and intent-preserving merge.

2.3 Protocol State Machines and Session Types

Protocol state machines, also known as typestate or session types , specify valid sequences of operations on a resource. CRABS extends typestate to a decentralized, CRDT-based setting where multiple participants can observe and verify protocol compliance. The resource locking protocol in CRABS is a direct application of typestate to resource scarcity.

2.4 Self-Sovereign Identity

Self-Sovereign Identity (SSI) is a paradigm where individuals control their own digital identities without relying on central authorities. Verifiable Credentials (VCs) and Decentralized Identifiers (DIDs) are key building blocks. CRABS's Attribute Machine provides a cryptographic foundation for SSI, where attributes are self-asserted or verified by peers, and all changes are recorded in an auditable transition log.

2.5 Capability-Based Security

Capability systems grant access based on possession of a capability token rather than identity. CRABS's capability vault — where ABE-encrypted ECDSA signing keys serve as capabilities — applies this principle to the attribute-based setting.

3. The CRABS Protocol

3.1 Architecture Overview

CRABS consists of three layers:

Layer 3: Application State Machines (Treasury, Documents, Voting, etc.) Each is a CRABS state machine with its own policies
Layer 2: The Attribute Machine State = all users + their attributes Operations = grant_role, revoke, self_assert, verify Policies = who can change what
Layer 1: Cryptographic Primitives ABE (CP-ABE via openabe-c) + Pluggable Signatures + SHA-256 CRDT/OT merge rules + Protocol state machines

The Attribute Machine is the foundation. Application machines reference its state for authorization decisions. All machines follow the same protocol.

3.2 Data Model

The state of a CRABS machine is a **bag of typed data items**:

```
State = {
  version: uint64,
  items: {
    "<name>": DataItem,
    ...
  },
  policies: {
    "<operation>": "<abe_policy_expression>",
```

```

    ...
  },
  lock_manager: LockManager,
  triggers: { ... },
  log: TransitionLog,
  config: MachineConfig
}

```

Each data item has a type, a CRDT merge strategy, a protocol state (for resource types), and a set of invariants. The type system includes basic CRDTs (counters, sets, registers, RGA) and OT/CRDT hybrids (ordered sets, rich documents, trees).

3.3 The Attribute Machine

The Attribute Machine is a CRABS state machine whose state is the set of all users and their attributes:

```

AttributeMachine State = {
  users: {
    "alice": {
      attributes: {
        role: "writer",
        dept: "engineering",
        phone: "555-0123",           // Self-asserted
        name: "Alice Smith"         // Verified by HR
      },
      keys: { ... },                // Multiple signature keys
      status: "active",
      key_version: 42
    }
  },
  policies: {
    "grant_role":      "role:admin",
    "revoke_role":     "role:admin",
    "verify_identity": "role:admin OR role:verifier",
    "self_assert":     "role:member",
    "change_policy":   "role:admin AND weight >= 3"
  },
  triggers: { ... }
}

```

Key insight: The Attribute Machine follows the *same protocol* as any other CRABS machine. There is no privileged "authority" — only protocol rules.

3.4 Key Derivation and Versioning

ABE keys are derived from the current state of the Attribute Machine:

```
KeyEnvelope = {  
  user_id: "alice",  
  state_version: 42,  
  attributes_hash: SHA256(alice_attrs),  
  sk_abe: ABE_keygen(MSK, "role:writer|dept:engineering|...")  
}
```

When attributes change, the state version increments, and users request key refreshes. Old keys are automatically invalidated by version mismatch.

4. OT/CRDT Hybrid Types

4.1 Motivation

Basic CRDTs support counters, sets, and registers but cannot represent ordered structures with move, swap, or reorder operations. For example, an OR-Set cannot express "move element from position 5 to position 2" — it can only add and remove. RGA supports insert and delete at positions but not move or swap.

OT/CRDT hybrid types extend CRDTs with the expressiveness of operational transformation while preserving convergence guarantees.

4.2 Coordinate Spaces

Every OT/CRDT hybrid type maintains two coordinate spaces:

```
Internal coordinates: The full structure, including deleted elements.  
Visible coordinates: What users see and reference in operations.
```

The mapping between spaces is maintained by a **position map** — a balanced BST that tracks all deleted and inserted positions:

```
xi(map, internal_pos) → visible_pos      // Internal → visible  
xi_inv(map, visible_pos) → internal_pos  // Visible → internal
```

4.3 The Transform Matrix

Every OT/CRDT type defines a transform matrix specifying how each operation type transforms against every other:

	INSERT(j)	DELETE(j)	MOVE(j→k)	SWAP(j,k)
INSERT(i)	priority compare	shift_left if i > j	shift_left if i > j	shift_left if i > min(j,k)
DELETE(i) if i >= j	shift_right	no_conflict	adjust	adjust
MOVE(i→l) if i >= j if i > j	shift_right	shift_left	composite	composite
SWAP(i1,i2) if i1 >= j if i1 > j	shift_right	shift_left	composite	composite

4.4 Priority-Based Deterministic Ordering

Concurrent inserts at the same position are ordered deterministically by priority:

```
priority = (node_timestamp << 48) | (node_id_hash & 0xFFFF)
```

Higher priority inserts before lower priority at the same position. Priority is unique across all nodes (node_id ensures uniqueness).

4.5 The Merge Algorithm

Algorithm: OT_CRDT_MERGE

Input:

item_a – Local replica
item_b – Remote replica

1. new_ops = item_b.op_log \ item_a.op_log
2. merged = copy(item_a)
3. for each op in new_ops:
4. for each existing_op in merged.op_log:
5. if concurrent(existing_op, op):
6. op = TRANSFORM(op, existing_op)
7. merged.op_log[i] = TRANSFORM(existing_op, op)
8. APPLY(merged, op)

```
9.     merged.op_log.append(op)
10. return merged
```

Property: After transform, applying operations in any order gives the same result — convergence without coordination, with intent preservation.

4.6 Defined Types

Type ID Name Operations Use Case `0x10` `OT_ORDERED_SET` insert, delete, move, swap Playlists, ranked lists `0x11` `OT_DOCUMENT` insert_text, delete_range, move, merge, split, style Collaborative editing `0x13` `OT_TREE` insert_node, delete_node, reparent, reorder File systems, org charts

5. Threshold Triggers

5.1 Motivation

In standard ABE, attributes are static — they are issued at key generation time and change only through explicit authority action. However, many applications require **automatic attribute issuance** based on state thresholds: revealing uploader contact info when flags on a video exceed a threshold, restricting upload privileges when reputation drops, or escalating support tickets that remain unresolved.

5.2 Trigger Definition

```
ThresholdTrigger = {
    trigger_id: string,
    condition: string,           // e.g., "video_abc.flags >= 10"
    condition_ast: ConditionNode,
    effect: TriggerEffect,
    cooldown_ms: uint64,
    last_triggered_at: uint64,
    one_shot: bool,
    enabled: bool
}

TriggerEffect = {
    type: ISSUE_ATTRIBUTE | CREATE_TRIGGER | DELETE_TRIGGER | CHANGE_POLICY,
    issue_attribute: string,    // e.g., "flag_threshold_met:video_abc"
    target_role: string,       // e.g., "moderator"
    duration_ms: uint64        // How long the attribute is valid
}
```

5.3 The Condition Language

```
<condition> ::= <or_expr>
<or_expr>   ::= <and_expr> ("OR" <and_expr>)*
<and_expr>  ::= <comparison> ("AND" <comparison>)*
<comparison> ::= <path> <op> <value>
               | <path> "CONTAINS" <value>
               | <path> "CONTAINS_ANY" "(" <value> ("," <value>)* ")"
               | <path> "CONTAINS_ALL" "(" <value> ("," <value>)* ")"
```

5.4 The Trigger Engine

Triggers are evaluated **after every state transition**:

```
Algorithm: PROCESS_TRIGGERS

1. PRUNE_EXPIRED_TEMPORARY_ATTRIBUTES(attribute_machine)
2. for each trigger in state.triggers:
3.     if not trigger.enabled: continue
4.     if in_cooldown(trigger): continue
5.     if EVALUATE_CONDITION(state, trigger.condition_ast):
6.         ISSUE_TEMPORARY_ATTRIBUTE(
7.             attribute_machine,
8.             trigger.effect.issue_attribute,
9.             trigger.effect.target_role,
10.            trigger.effect.duration_ms
11.        )
12.        trigger.last_triggered_at = now()
13.        LOG(state, {type: "__trigger_fired__", ...})
```

5.5 Progressive Disclosure

Multiple thresholds enable progressive disclosure of increasingly sensitive data:

```
Trigger 1: flags >= 10 → issue "flag_threshold_10" to moderator
                    → can decrypt uploader's username

Trigger 2: flags >= 50 → issue "flag_threshold_50" to moderator
                    → can decrypt uploader's email

Trigger 3: flags >= 100 → issue "flag_threshold_100" to admin
                    → can decrypt uploader's phone
```

Each level of contact information is encrypted under a different policy, and the attributes are only issued when the corresponding threshold is met.

6. Cryptographic Agility

6.1 Motivation

The base CRABS protocol specifies ECDSA with secp256k1 as the sole signature scheme. This creates a hard dependency on a single cryptographic primitive, which is undesirable for post-quantum migration, performance diversity, and regulatory compliance.

6.2 The VTable Interface

Every signature scheme implements a common interface:

```
SignatureVTable = {
    scheme_id: uint8,
    name: string,
    properties: SchemeProperties,
    generate_keypair: function(pk, pk_len, sk, sk_len) → int,
    sign: function(sk, sk_len, msg, msg_len, sig, sig_len) → int,
    verify: function(pk, pk_len, msg, msg_len, sig, sig_len) → int,
    verify_batch: function(...) → int,      // Optional
    aggregate_signatures: function(...) → int // Optional
}
```

6.3 Supported Schemes

Scheme	ID	Key Size	Sig Size	Post-Quantum	Batch	ECDSA secp256k1
Ed25519	0x01	32 B	64 B	No	No	No
BLS12-381	0x03	48 B	96 B	No	Yes	No
Dilithium 3	0x05	1952 B	3293 B	Yes	No	No
Falcon	0x0B	897 B	666 B	Yes	No	No

6.4 Multi-Key Users

Users can register multiple keys for different schemes:

```
User = {
    ...,
    keys: {
        "ecdsa_main": {scheme: ECDSA_SECP256K1, public_key: ..., is_active: true},
```

```

    "ed25519_mobile": {scheme: ED25519, public_key: ..., is_active: true},
    "dilithium_hsm": {scheme: DILITHIUM_3, public_key: ..., is_active: true}
  },
  default_key_id: "ecdsa_main"
}

```

6.5 Policy Integration

Policies can reference signature schemes:

```

"role:admin AND sig_scheme:ed25519"
"role:admin AND (sig_scheme:ecdsa_secp256k1 OR sig_scheme:dilithium_3)"

```

This enables graceful migration between schemes during transition periods.

7. Idempotent Operations

7.1 Motivation

Byte-level idempotency (UUID deduplication) prevents the same operation bytes from being applied twice. However, many applications require **semantic idempotency** — preventing the same *logical action* from being performed twice, even if the operation bytes differ.

7.2 DedupSpec

```

DedupSpec = {
  type: PER_USER | GLOBAL | CUSTOM,
  tracker_path: string,      // For PER_USER: ONE_SHOT_SET path
  flag_path: string,        // For GLOBAL: ONE_SHOT_FLAG path
  condition: string,        // For CUSTOM
  update: StateMutation,
  rejection_message: string
}

```

7.3 New Data Types

ONE_SHOT_SET (Type 0x08): A set where each element can be added at most once. CRDT merge is union — once an element exists in any replica, it exists in all.

ONE_SHOT_FLAG (Type 0x09): A boolean flag that can transition from `false` to `true` exactly once. CRDT merge is OR — once true in any replica, always true.

7.4 Patterns

Pattern Dedup Type Tracker Example One vote per user `PER_USER` `ONE_SHOT_SET`
`proposal_42.voters` Execute once `GLOBAL` `ONE_SHOT_FLAG` `proposal_42.executed` First-past-the-post `GLOBAL` `ONE_SHOT_FLAG` `username_claims.alice123` Single-use token `GLOBAL`
`ONE_SHOT_FLAG` `codes.SUMMER2024`

8. Tombstone Management

8.1 The Tombstone Problem

Every CRDT that supports deletion accumulates tombstones — metadata that must persist to ensure correct convergence. For OR-Sets, tombstones are removed element tags. For RGA, tombstones are deleted nodes in the linked list. For OT/CRDT types, tombstones are nodes in the position map BST.

Without management, tombstones grow linearly with edit count, not document size. A document with 5,000 visible characters and 100,000 edits has ~100,000 tombstones.

8.2 Unified Compaction Framework

Every tombstone-bearing type implements a compaction vtable:

```
CompactionVTable = {  
    extract_visible: function(item) → visible_value,  
    rebuild_from_visible: function(visible) → fresh_item,  
    count_tombstones: function(item) → uint64,  
    count_visible: function(item) → uint64,  
    compaction_safe: function(item, state) → bool  
}
```

8.3 Compaction Algorithm

```
Algorithm: COMPACT_ITEM  
  
1. vtable = GET_COMPACTION_VTABLE(item.type)  
2. visible = vtable.extract_visible(item)
```

```

3. compacted = vtable.rebuild_from_visible(visible)
4. compacted.last_compaction_time = now()
5. LOG(state, {type: "__compaction__", ...})
6. return compacted

```

8.4 Safety

Compaction is safe only when all peers have observed all operations prior to the compaction point. This is determined by vector clock comparison:

```

for each peer in state.peers:
    for each node in state.nodes:
        if peer.vector_clock[node] < state.max_sequence[node]:
            return NOT_SAFE
return SAFE

```

8.5 Types Covered

Type Tombstone Source Compaction Effect OR-Set Removed element tags Elements get fresh tags 2P-Set Remove set remove_set emptied (caveat: re-add allowed) RGA Deleted nodes Linked list rebuilt OT types Position map nodes Coordinate space reset

9. Security Analysis

9.1 Threat Model

We assume the following adversary capabilities:

- **Network adversary:** Can observe, delay, reorder, and drop messages
- **Byzantine nodes:** Up to f nodes may behave arbitrarily
- **Compromised users:** Can attempt to use stale keys, forge signatures, or collude

We do not defend against compromise of the MSK (though threshold MSK schemes are future work) or quantum attacks on bilinear pairings.

9.2 Security Properties

Property Mechanism Guarantee **Authorization** ABE policy verification Only users with matching attributes can perform operations **Authentication** Pluggable signatures Operations bound to a specific

key and scheme **Non-repudiation** Signed log entries Past operations cannot be denied **Collusion resistance** ABE key randomization Two users cannot combine keys **Replay protection** UUID + Lamport clocks Old messages cannot be replayed **Liveness** Lock timeout + force-unlock No resource locked indefinitely **Privacy (Mode B)** Optional signer omission Verifier learns only that *some* authorized user signed **Convergence** CRDT/OT merge rules All correct nodes reach same state **Intent preservation** Transform functions Concurrent edits preserve user intent

9.3 Threat Mitigations

Threat Mitigation **Trigger spam** Cooldown period, one-shot triggers **False triggers** Weighted counters, multi-signature thresholds **Scheme downgrade** sig_scheme included in signed data **Key compromise** Key revocation protocol, key versioning **Tombstone bloat** Configurable compaction, emergency thresholds **Compaction safety** Vector clock synchronization check

10. Use Cases

10.1 Decentralized Autonomous Organizations (DAOs)

CRABS provides a natural foundation for DAOs where membership is defined by attributes rather than token holdings. The Attribute Machine governs roles, and application machines govern treasury, voting, and proposals. Threshold triggers enable automatic escalation, and OT/CRDT types enable collaborative document editing for proposals.

10.2 Collaborative Content Platform

A YouTube-like platform where:

- Video metadata is stored in OT_DOCUMENT items (collaborative descriptions)
- View counts use G-Counters (tamper-proof)
- Flags use PN-Counters with threshold triggers for automatic moderation
- Uploader contact info is encrypted under threshold-gated policies
- Comments use OT_ORDERED_SET for threaded discussions

10.3 Self-Sovereign Identity

The Attribute Machine enables a fully self-sovereign identity system where users control their own attributes within policy constraints. Verifiers can attest to claims, and all changes are logged and auditable.

10.4 Supply Chain Management

Each shipment is a resource with a protocol state machine governing custody transfers. Different organizations (customs, carriers, warehouses) have different roles, and the protocol ensures that custody transfers are atomic and auditable.

11. Limitations and Future Work

11.1 Limitations

Limitation Impact Mitigation **MSK single point of compromise** Attacker can forge any attribute Threshold MSK (future work) **No post-quantum ABE** Pairings vulnerable to Shor's algorithm Lattice-based ABE (future work) **OT transform complexity** $O(N^2)$ merge for concurrent operations Log pruning, compaction **Tombstone growth** Linear with edit count Unified compaction framework **No global consensus** Temporary divergence possible CRDT merge guarantees eventual convergence

11.2 Future Work

Threshold MSK: Distributing the MSK across n nodes, requiring k to sign, would eliminate the single point of compromise.

Post-Quantum CRABS: Porting to lattice-based ABE would provide post-quantum security. The protocol architecture is agnostic to the underlying cryptographic primitives.

Hierarchical Attribute Machines: Organizations with multiple departments could have cross-machine trust relationships.

Formal Verification: The protocol's safety properties could be formally verified using Coq or Isabelle/HOL.

Economic Incentives for Force-Unlock: A small reward for force-unlocking expired locks would ensure liveness in practice.

OT/CRDT Type Generator: A code generation framework for deriving transform matrices from type definitions.

12. Conclusion

CRABS addresses the fundamental limitation of Attribute-Based Encryption — the central authority problem — by treating attributes as dynamic, protocol-governed state rather than static cryptographic material. The Attribute Machine, a CRABS state machine whose state *is* the attribute universe, eliminates the need for a trusted authority while preserving ABE's cryptographic guarantees.

By extending this foundation with OT/CRDT hybrid types, threshold triggers, cryptographic agility, and unified tombstone management, CRABS provides a comprehensive framework for decentralized authorization that is:

- **Self-sovereign** — Users control their own attributes within policy constraints
- **Expressive** — OT/CRDT types support move, swap, reorder, and intent-preserving merge
- **Convergent** — CRDT merge ensures eventual consistency without consensus
- **Responsive** — Threshold triggers bridge state changes with cryptographic access
- **Future-proof** — Pluggable signatures enable post-quantum migration
- **Sustainable** — Unified compaction manages tombstone growth

The protocol is implementable today with existing libraries (openabe-c for ABE, OpenSSL for ECDSA, libsodium for Ed25519, liboqs for post-quantum schemes) and is suitable for applications ranging from DAOs and collaborative content platforms to supply chain management and self-sovereign identity.

References

- A. Sahai and B. Waters, "Fuzzy Identity-Based Encryption," EUROCRYPT 2005.
- V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-Based Encryption for Fine-Grained Access Control of Encrypted Data," CCS 2006.
- J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-Policy Attribute-Based Encryption," S&P 2007.
- M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-Free Replicated Data Types," SSS 2011.

- C. Ellis and S. Gibbs, "Concurrency Control in Groupware Systems," SIGMOD 1989.
- R. Levien, "Operational Transformation and CRDT Synthesis," 2016. <https://github.com/google/ot-crdt-papers>
- M. Chase, "Multi-Authority Attribute Based Encryption," TCC 2007.
- A. Lewko and B. Waters, "Decentralizing Attribute-Based Encryption," EUROCRYPT 2011.
- D. Nichols, P. Curtis, M. Dixon, and J. Lamping, "High-Latency, Low-Bandwidth Windowing in the Jupiter Collaboration System," UIST 1995.
- R. Levien, "Replicated Growable Array (RGA)," 2016. <https://github.com/xi-editor/xi-editor>
- R. DeLine and M. Fähndrich, "Typestates for Objects," ECOOP 2004.
- K. Honda, V. Vasconcelos, and M. Kubo, "Language Primitives and Type Discipline for Structured Communication-Based Programming," ESOP 1998.
- C. Allen, "The Path to Self-Sovereign Identity," 2016.
- J. Dennis and E. Van Horn, "Programming Semantics for Multiprogrammed Computations," CACM 1966.